ENTERPRISE SDN CONTROLLER

Consolidated Technical Blueprint & Deployment Architecture

High-Availability Microservice Architecture, Automation Framework,
and Engineering Review

Date: June 25, 2026
Prepared for: Hamza ZERTA
Prepared by: Mostafa Faouzi

Program: Master 2 RES, Sorbonne University

Table of Contents

Part 1 — Architectural Rationale

1.1 From Standalone VMs to Microservices

The reference architecture replaces a legacy footprint of 5 standalone, dedicated virtual machines (3 application nodes, 1 core load balancer, and 1 PNETLab host) with a converged 3-node SUSE RKE2 Kubernetes cluster paired with a single dedicated off-cluster simulation engine.

The guiding principle is to separate stateless from stateful workloads, so each class of load can scale and be restored independently — rather than scaling an entire VM vertically every time one component spikes.

Why Microservices over Standalone VMs

- **Decoupled Horizontal Scaling —** individual components scale independently using Horizontal Pod Autoscalers (HPA) or event-driven controllers like KEDA, based on live queue depth or API request volume.
- **Declarative Infrastructure —** the entire fabric plane (ingress, secrets, storage paths) is version-controlled YAML, eliminating configuration drift at the host OS level.
- **High-Availability Self-Healing —** a node fault triggers automatic pod rescheduling onto surviving infrastructure within seconds, without dropping data paths.

1.2 High-Level Deployment Flow

Layer	Role
Rancher Management Plane	Cluster lifecycle, Kubernetes RBAC, Helm catalogue, infra monitoring (Prometheus + Grafana)
Ingress Layer	MetalLB (Layer-2 VIP) + NGINX Ingress Controller — replaces the dedicated HAProxy/NGINX VM
Node Pool “stateless”	Deployments: fastapi-gateway, celery-workers, gnmi-collector, config-compliance-mgr
Node Pool “stateful”	StatefulSets: PostgreSQL (CloudNativePG/Zalando), Redis Sentinel
Storage Layer	Longhorn (CSI block storage) + MinIO (S3-compatible: backups, snapshots, exports)
Off-cluster	Dedicated PNETLab VM — 20× Dell OS10 virtual switches, nested virtualization

Request flow: Client/Switch → MetalLB VIP → NGINX Ingress (TLS termination, JWT auth) → fastapi-gateway service → dispatch to celery-workers (Redis queue), gnmi-collector (PNETLab via network), or config-compliance-mgr (snapshots → MinIO) → PostgreSQL / Redis Sentinel StatefulSets on Longhorn PVCs.

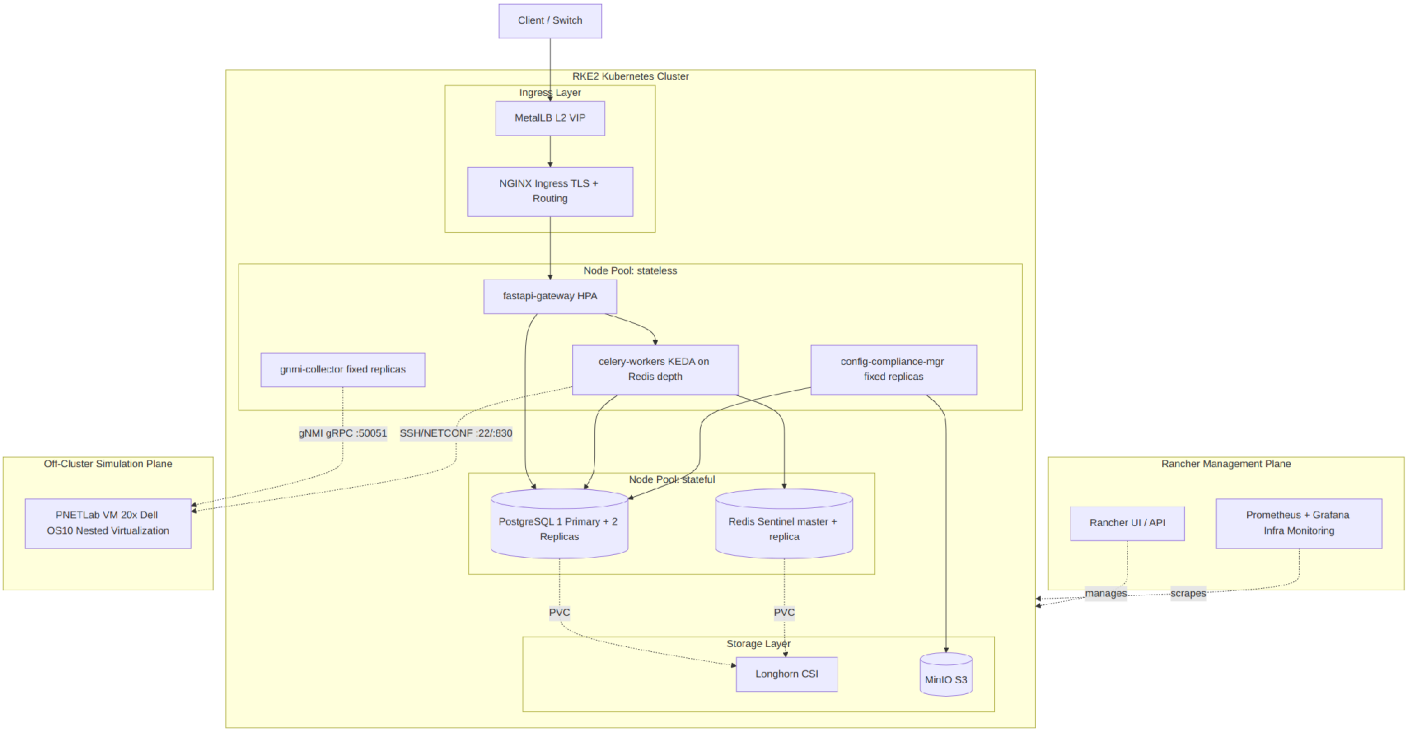

Figure 1 — Overall deployment architecture: Rancher management plane, RKE2 cluster (ingress, stateless/stateful node pools, storage), and the off-cluster PNETLab simulation plane.

1.3 Why a StatefulSet, Not a Deployment, for Data Tiers

A StatefulSet guarantees a stable network identity (predictable hostname) and a persistent attachment to its own volume — indispensable for a database, unlike a Deployment where pods are freely interchangeable.

1.4 Storage Strategy

System	Usage	Rationale
Longhorn	StorageClass for Postgres & Redis PVCs (block storage)	Native to the Rancher/SUSE ecosystem, one-click install, automatic cross-node replication, integrated snapshots — no external SAN required
MinIO	S3-compatible store: Postgres backups (pgBackRest/WAL-G, PITR), archived config snapshots, reporting exports	De-facto standard for this data class; reusable for future needs (e.g. archiving historical config payloads as JSON)

Velero is distinct from the Postgres backup path: it preserves Kubernetes manifests, PVC definitions, and secrets to MinIO. Without it, a cluster incident (configuration loss, operator error) would destroy not just data but the entire deployment definition — Velero restores full cluster state, not just business data.

Recommendation — schedule restore drills

Velero and pgBackRest are the right tools, but a backup that has never been restored is unverified. Define a recurring restore-test cadence (e.g. monthly) covering both cluster-state restore (Velero) and Postgres PITR restore.

1.5 PNETLab: Why It Stays Off-Cluster

PNETLab requires nested virtualization (VT-x/AMD-V) to run 20 virtual Dell OS10 switches — this is not a workload suited to a standard Kubernetes pod (risky in privileged mode, no clean native support).

- **Current choice:** a dedicated VM, separate from the Rancher cluster. The gnmi-collector and southbound drivers (arista_eos.py, nokia_srlinux.py, dell_os10.py) connect to it over the network.
- **Future option:** migrate to Harvester (Rancher's KubeVirt-based product) to run these VMs inside the cluster, unifying VM and workload management under one Rancher interface. Not adopted yet — migration complexity isn’t justified before the application cluster is stabilized.

Part 2 — Infrastructure Sizing

2.1 Resource Allocation Matrix

Component	Nodes	vCPU	RAM
PNETLab Host (Simulation Plane)	1	32–40 vCPU	64–80 GB
RKE2 Cluster Nodes (Application Plane)	3	6–8 vCPU / node	16 GB / node
Aggregate Footprint	4 VMs	~50–64 vCPU	~112–128 GB

Storage: 150 GB NVMe for the PNETLab host (Ubuntu Server LTS / PNETLab Emulation Engine Core); 100 GB NVMe per RKE2 node (Ubuntu 24.04 LTS / RKE2 / Rancher Control Agent) — roughly 450 GB NVMe in aggregate.

2.2 Simulation Tier Engineering Constraints

- **Per-switch footprint (Dell OS10.5.5 / 10.5.4):** 1–2 vCPUs and 3–4 GB RAM per virtualized switch, to reliably complete kernel initialization and host RESTCONF, BGP, LLDP, and gNMI daemons.
- **Virtual disks:** standard QCOW2 sparse volumes (sataa.qcow2, virtiob.qcow2, virtioc.qcow2) avoid pre-allocating large physical blocks.
- **Mandatory nested virtualization:** PNETLab runs switches as secondary QEMU VMs, so nested virtualization must be enabled at the root hypervisor (ESXi, Proxmox, bare-metal cloud). Without it, QEMU falls back to software emulation, causing CPU starvation and boot failures.

2.3 Node Pool Detail — Stateless

Deployment	Role	Scaling	Notes
fastapi-gateway	Central API gateway (main.py): admin routing, policy, telemetry, config	HPA on CPU / request count	Single synchronous entry point
celery-workers	Async execution: southbound provisioning, Ansible/config-lifecycle jobs, gNMI event processing	KEDA ScaledObject on Redis Streams queue depth	Queue depth is more relevant than CPU during 20+ switch compliance audits
gnmi-collector	Persistent gNMI Subscribe sessions (LLDP, BGP, telemetry)	Fixed replicas / manual scaling	Each pod can own a subset of switches
config-compliance-mgr	Snapshotting, golden-config audits, rollback & blast-radius triggering	Fixed replicas	Low simultaneous instance volume

Rationale for 4 separate Deployments rather than one monolith: each has a distinct load profile (synchronous API vs. async jobs vs. long-lived connections vs. periodic jobs), so each must scale independently without wasting resources on the others.

2.4 Node Pool Detail — Stateful

StatefulSet	Role	HA Detail
postgresql	Shared database (see data model, §2.6)	Operator-managed (CloudNativePG or Zalando): 1 primary + 2 replicas, automatic failover, PITR
redis-sentinel	Event bus (Redis Streams) + cache	1 master + 1 replica + Sentinels monitor and auto-promote a new master on failure

Part 3 — Switch Configuration Lifecycle

Managed switches traverse a strict internal state machine: DiscoveredRaw → baseline snapshot → CompliantActive → periodic audits (Pass/Fail) → ConfigurationDrifted on manual drift → TriggerRollback → back to CompliantActive.

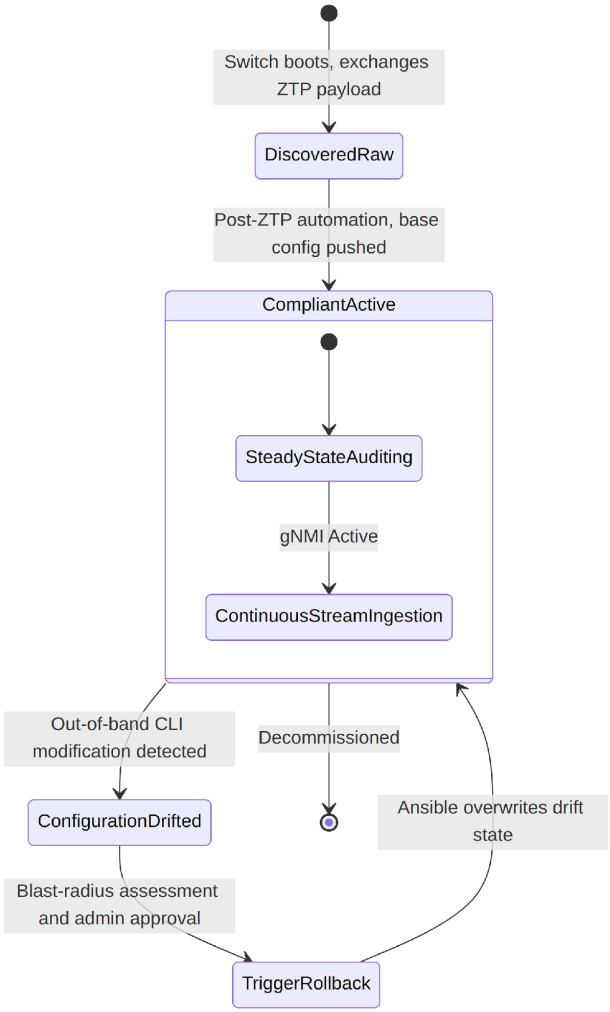

Figure 2 — Switch configuration lifecycle state machine.

3.1 DiscoveredRaw — Zero-Touch Onboarding

1. Switch boots and broadcasts a DHCP Discover request with DHCP Option 67 (bootstrap script URL).
2. DHCP server responds with an IP and the option-67 URL.
3. The switch executes the script and issues an unauthenticated POST to /api/v5/discovery/on-boarding-ingestion (MAC, serial, OS, vendor) — unauthenticated because the ZTP network path itself validates the origin.
4. The controller upserts a record in ztp_discovery_pool and returns 202 Accepted.

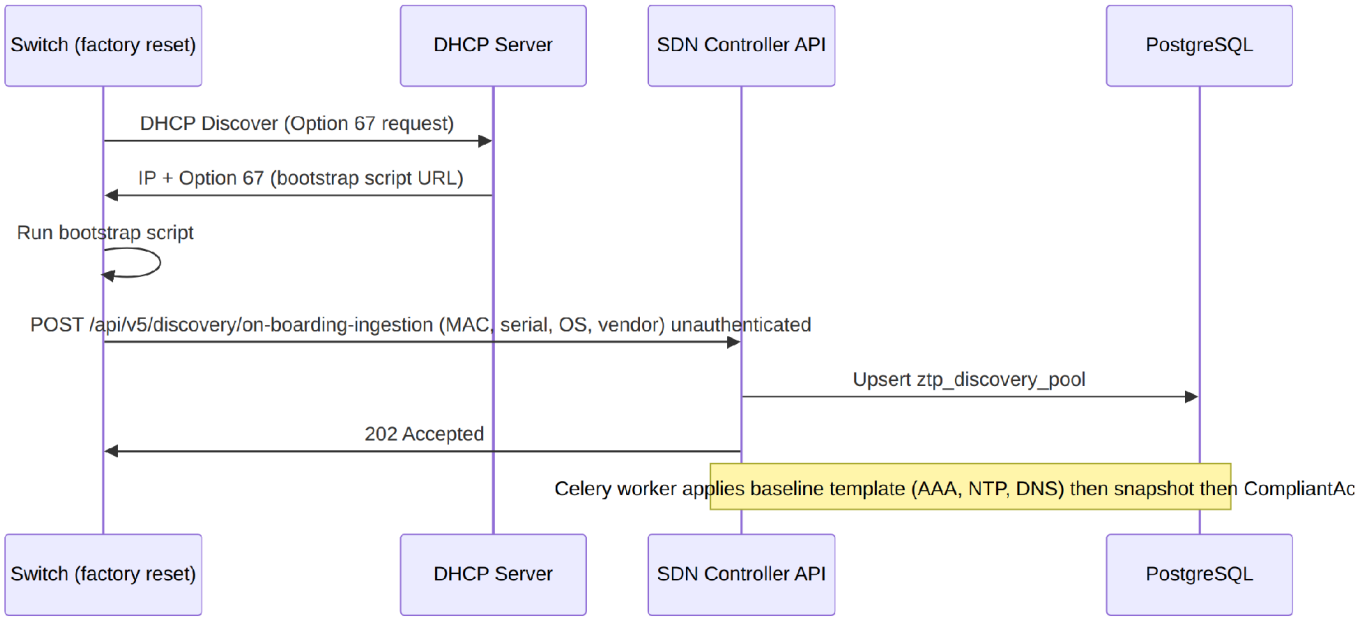

Figure 3 — Zero-Touch Provisioning (ZTP) onboarding sequence.

3.2 CompliantActive

A Celery worker applies the baseline template (AAA, corporate user creation, NTP, DNS), takes a clean configuration snapshot into the append-only config_snapshots table, and marks the asset CompliantActive.

3.3 ConfigurationDrifted

If an engineer bypasses the controller API (local console or direct SSH) and modifies a parameter — e.g. an active VLAN string or interface state — the background config-compliance-mgr daemon flags the mismatch on its next periodic audit pass, and the switch moves to ConfigurationDrifted.

Audit vectors checked: NTP target (192.168.100.1), DNS resolver target (8.8.8.8), and AAA local management authentication (must be explicitly enabled — critical flag).

3.4 TriggerRollback

On drift detection or manual trigger, the controller pulls the last verified configuration blob from the snapshot repository; a Celery worker executes the recovery playbook, rewrites the switch configuration, and returns the asset to CompliantActive.

3.5 Blast-Radius Protection

Target	Impact Index	Approval Path
Leaf switch	1 device	Auto-approved — dispatched to Celery immediately
Spine switch	6 devices (cross-fabric)	Blocked from direct execution — frozen queue, Four-Eyes Approval required from a Platform Admin

Recommendation — unify the blast-radius threshold

The spine threshold is framed two ways across source material: “impact index 6” vs. “blast radius > 2 → Four-Eyes mandatory.” Resolve to a single numeric rule (e.g. “impact index > 2 devices requires Four-Eyes Approval”) and make it a configurable value rather than a hardcoded constant, so the policy can be tuned without a code change.

Recommendation — protect the ZTP onboarding endpoint

/api/v5/discovery/on-boarding-ingestion is intentionally unauthenticated by design (the ZTP network path itself validates origin). Add IP-based rate limiting at the ingress layer so a misbehaving or spoofed device cannot flood the ztp_discovery_pool table.

Recommendation — idempotency on the streaming path

Redis Streams plus Celery retries can redeliver a task. Southbound Ansible playbooks are idempotent by design, but the gNMI streaming/event side should confirm duplicate event handling will not double-write topology_edges.

Part 4 — Northbound Policy Pipeline

A policy intent submitted via POST /api/v5/orchestrator/policy-enforcement must clear a strict 4-stage pipeline before reaching the database or hardware.

Stage	Check	Failure Mode	Detail
1. Syntax Validation	Schema & IP network sanity (Pydantic, ipaddress.ip_network)	400 Bad Request	Malformed CIDR / field types rejected immediately
2. Tenant Boundary Isolation	tenant_vrfs / ipam_subnets cross-checks	400 Bad Request	Hardcoded in the pipeline core — holds even if a token claim is compromised
3. Topology & VLAN Collision	topology_nodes / topology_edges graph checks	400 Bad Request	Confirms target switches are online & registered; VLAN ID not already mapped to another VRF on that fabric
4. Dry-Run Diff Engine	Southbound driver generates exact CLI diff	N/A (preview)	dry_run=true returns the diff only; dry_run=false commits to DB and dispatches to Redis Streams

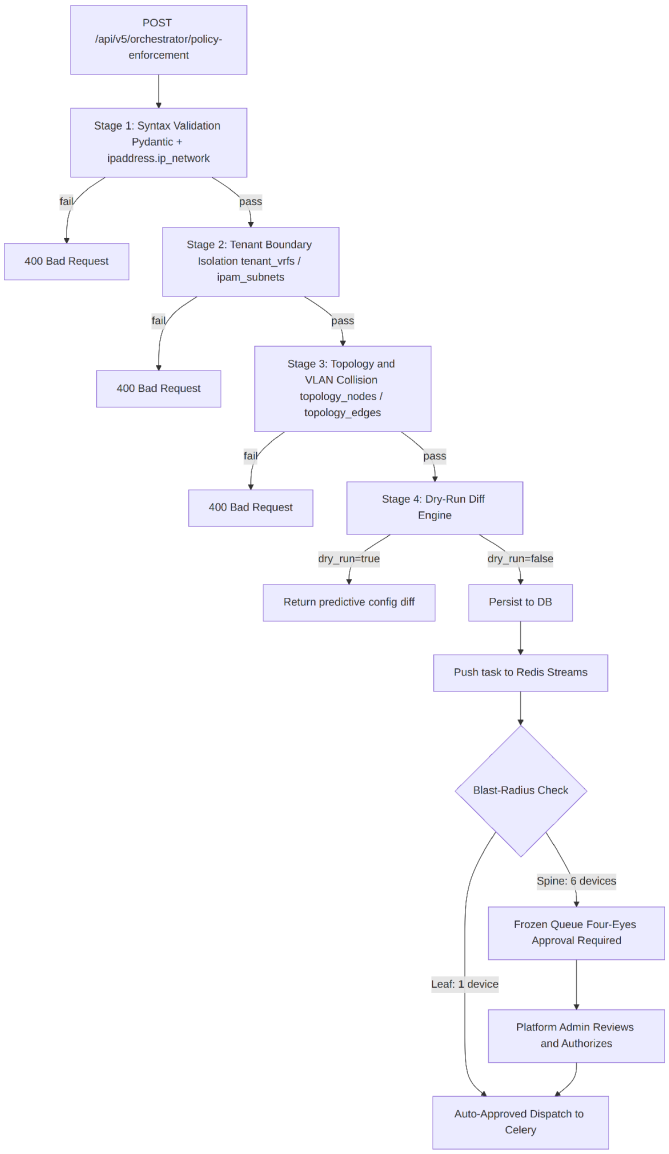

Figure 4 — 4-stage northbound policy pipeline, including the blast-radius approval branch.

4.1 Hybrid Southbound Automation

The automation plane combines two engines under a shared Celery worker layer:

- **Asynchronous Python Engine —** persistent gNMI streaming over gRPC (port 50051) via pygnmi, feeding the live topology graph. Subscribes to /system/lldp for link-layer discovery and extracts MAC/ARP tables for endpoint tracking (discovered_endpoints).
- **Idempotent Ansible Engine —** ansible-runner invoked from Celery for confirmed intents. Uses Jinja2 templates and native vendor modules (e.g. dellemc.os10.os10_vlan) over SSH (22) or NETCONF (830), applying only the command deltas needed. A scheduled audit_config.yml playbook drives the continuous compliance check.

Ansible is intentionally not used for streaming telemetry — its short-lived, transaction-focused SSH model is structurally inefficient for long-running subscriptions.

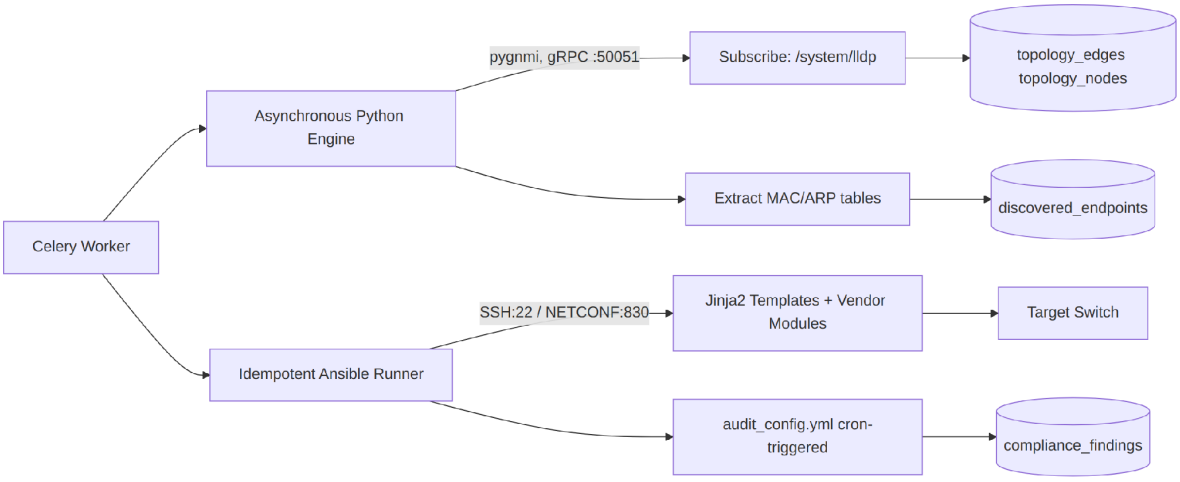

Figure 5 — Hybrid southbound automation: asynchronous Python/gNMI engine vs. idempotent Ansible engine.

Recommendation — dedicated test coverage for the pipeline

Given the blast-radius stakes, Stage 2 (tenant boundary isolation) and Stage 3 (topology/VLAN collision) deserve dedicated unit and integration tests before any production traffic, alongside coverage for the southbound drivers.

Part 5 — Data Model & Northbound API

5.1 PostgreSQL Schema Groups

Table Group	Role
users, tenants, tenant_vrfs	Identity and multi-tenant isolation
fabrics, fabric_blueprints	Physical fabric definitions and templates
ipam_subnets, ipam_ip_allocations	VRF-aware IP address management
ztp_discovery_pool, switches	Inventory and onboarding
topology_nodes, topology_edges	LLDP/BGP topology graph
discovered_endpoints	Learned MAC/IP endpoints
telemetry_metrics, telemetry_metadata	Operational metric series
config_snapshots	Append-only configuration archive
compliance_runs, compliance_findings	Audit results

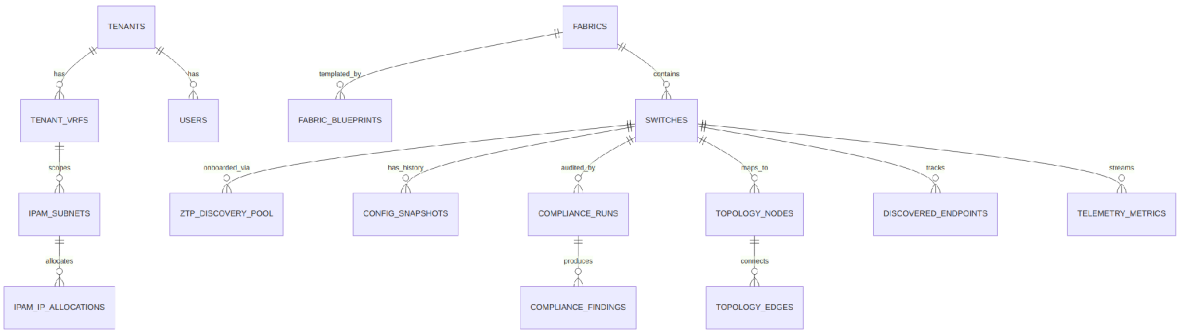

Figure 6 — Simplified entity-relationship diagram of the core PostgreSQL schema.

5.2 Northbound API Reference

Endpoint	Method	Function
/api/v5/auth/login	POST	Authentication → JWT
/api/v5/orchestrator/policy-enforcement	POST	Submit network intent (dry_run supported)
/api/v5/orchestrator/policy-reconciliation	POST	Remove allocation + rollback
/api/v5/discovery/on-boarding-ingestion	POST	ZTP ingestion (no auth)
/api/v5/visibility/snapshots	POST/GET	Configuration snapshot / history
/api/v5/visibility/rollback	POST	Rollback with blast-radius evaluation
/api/v5/visibility/compliance/run | /latest	POST/GET	Compliance audit

Endpoint	Method	Function
/api/v5/visibility/endpoints	GET	Learned endpoints
/api/v5/visibility/telemetry	GET	Telemetry series
/api/v5/admin/stats | /topology	GET	Stats and topology graph

5.3 Security & RBAC

Recommendation — trace requests end-to-end

Prometheus/Grafana cover cluster health, but consider structured application logs with a correlation ID per policy-enforcement request, so a failed Stage 2 or 3 rejection can be traced end-to-end without grepping multiple pods.

- JWT Bearer required for every write operation.
- Three roles: Platform Admin (full access), Tenant Operator (write, scoped to tenant), Tenant Auditor (read-only, scoped to tenant).
- Tenant isolation is enforced as early as Stage 2 of the policy pipeline — not only at the JWT layer.

⚠ Two distinct authorization layers

Application RBAC (JWT, tenant scoping) must not be confused with Kubernetes RBAC managed by Rancher (who can administer the cluster itself). These are two completely separate authorization planes.

Recommendation — explicit secrets management

Neither source document specifies where JWT signing keys, DB credentials, or device SSH/NETCONF credentials live. Use Kubernetes Secrets backed by Sealed Secrets or an external vault (HashiCorp Vault, or Rancher's built-in secret store) rather than plain manifests.

Recommendation — inter-service network policy

With 4 stateless Deployments and 2 StatefulSets, add Kubernetes NetworkPolicies so only celery-workers and config-compliance-mgr can reach Postgres directly — the gateway should not have a direct DB path that bypasses the pipeline.

Recommendation — plan the RBAC evolution path now

Three fixed roles is reasonable for v1, but design roles as named permission sets from the start, so migrating to granular (40+) permission-based RBAC later does not require a breaking schema change.

Part 6 — Functional Roadmap (Gap Analysis)

Gap assessment against a target perimeter comparable to a commercial NAC/network-tracking platform:

Capability	Status	Effort	Notes
BGP peering graph	⚠ Not implemented	Medium	Add a gNMI Subscribe on /network-instances/.../bgp
ISIS / MPLS / LDP topology	✖ Not implemented	High	Defer until a concrete business use case requires it
Compliance scoring & trending	⚠ Partial	Low	Enrich compliance_runs with an aggregated score field
Reporting & exports (CSV/XLSX)	✖ Not implemented	Low	Dedicated endpoint + pandas/openpyxl + Celery Beat scheduling
Granular RBAC (40+ permissions)	⚠ Partial (3 fixed roles)	Medium	Extend the JWT claim toward a permission-based model
Interactive frontend	⚠ To be built	High	React SPA + react-flow / cytoscape.js

Recommended sequencing: prioritize compliance scoring/trending and reporting/export first (low effort, high value), then BGP peering, with ISIS/MPLS/L2VPN only if a concrete operational need emerges.

Part 7 — Consolidated Technology Stack

Layer	Technology
Cluster orchestration	Rancher + RKE2 (Kubernetes)
Ingress	MetalLB + NGINX Ingress Controller
Autoscaling	HPA (Gateway), KEDA (Celery on Redis queue depth)
Database	PostgreSQL via operator (CloudNativePG / Zalando)
Event bus / cache	Redis Sentinel (StatefulSet)
Block storage (CSI)	Longhorn
Object storage (S3)	MinIO
Cluster backup	Velero
Postgres backup (PITR)	pgBackRest / WAL-G
Infra monitoring	Prometheus + Grafana (Rancher-integrated)
API Gateway	FastAPI (main.py)
Policy validation	Pydantic + custom 4-stage pipeline
Discovery & telemetry	gNMI (pygnmi), gnmi_discovery.py, metrics_collector.py
Config lifecycle	config_lifecycle.py
Job orchestration	Celery + Redis Streams
Southbound drivers	drivers/dell_os10.py, drivers/nokia_srlinux.py, drivers/arista_eos.py
Application auth	JWT Bearer (3 roles)
Network test environment	PNETLab (20× Dell OS10, dedicated VM, nested virtualization)

7.1 Target Project Directory Structure

```
enterprise-sdn-controller/
├── .github/
│   └── workflows/                # CI/CD pipelines for automating Docker image builds
├── k8s-infra/                    # Rancher & RKE2 Kubernetes Manifests
│   ├── metallb-config.yaml       # MetalLB Layer-2 VIP pool allocations
│   ├── longhorn-values.yaml      # StorageClass & replication helm values
│   ├── minio-values.yaml         # MinIO backup bucket definitions
│   └── apps/                     # Application deployment manifests
│       ├── ingress.yaml          # NGINX Ingress routing rules
│       ├── gateway-deploy.yaml   # FastAPI application service spec
│       ├── workers-deploy.yaml   # Celery automation microservice spec
│       ├── collector-deploy.yaml # gNMI streaming daemon spec
```

```
│       ├── postgres-spec.yaml    # High-availability PostgreSQL StatefulSet
│       └── redis-spec.yaml       # Redis Sentinel distributed cache spec
├── src/                          # Core Python Application Source Code
│   ├── main.py                   # FastAPI app entry point & 4-Stage Pipeline logic
│   ├── database.py               # SQLAlchemy connection & session manager
│   ├── auth/                     # Consumption Plane security definitions
│   │   ├── dependencies.py       # JWT verification & multi-tenant isolation handlers
│   │   └── models.py             # Security user database schemas
│   ├── models/                   # Relational database mappings (SQLAlchemy Models)
│   │   ├── network.py            # Switch, Fabric, and Topology schema graphs
│   │   ├── ipam.py               # Subnets and multi-tenant IPAM allocation schemas
│   │   └── compliance.py         # Compliance runs and config snapshot definitions
│   └── automation/               # Automated Network Control Core
│       ├── tasks.py              # Celery task wrappers (ZTP onboarding, audits)
│       ├── streaming/            # Live telemetry collectors
│       │   ├── metrics_coll.py   # CPU, memory, and port statistics processor
│       │   └── topology_disc.py  # Asynchronous LLDP topology engine (pygnmi)
│       ├── compliance/           # Config validation logic
│       │   ├── engine.py         # Golden-config validation loops (NTP/DNS/AAA)
│       │   └── blast_radius.py   # Blast-radius index risk calculators
│       └── drivers/              # Southbound protocol interfaces
│           ├── base.py           # Abstract Base Driver blueprint
│           ├── python_gnmi.py    # Native Python gNMI implementation
│           └── ansible_runner.py # Integration module for programmatically executing
playbooks
├── southbound-ansible/           # Southbound Provisioning Plane (Ansible Engine)
│   ├── ansible.cfg               # Core Ansible engine behavior settings
│   ├── inventory/
│   │   └── pnetlab_switches.yml  # Target definitions mapping to PNETLab switch instances
│   ├── group_vars/
│   │   ├── all.yml               # Global network variables (NTP targets: 192.168.100.1)
│   │   └── dell_os10.yml         # Shared Dell OS10 configuration variables
│   ├── roles/                    # Reusable network configuration building blocks
│   │   ├── base_provisioning/    # Baseline configuration primitives (AAA, DNS, NTP
options)
│   │   ├── vlan_vrf_config/      # Multi-tenant network isolation playbooks
│   │   └── rollback_handler/     # Configuration drift overwrite playbooks
│   ├── deploy_fabric.yml         # Main playbook orchestrating active fabric topology
shifts
│   └── audit_config.yml          # Automation playbook running periodic drift checks
├── Dockerfile.gateway            # Container recipe for building the API Gateway image
├── Dockerfile.worker             # Container recipe for building the Celery Worker image
├── Dockerfile.collector          # Container recipe for building the gNMI Collector image
├── requirements.txt              # Production Python package locks (FastAPI, Celery,
pygnmi)
└── README.md                     # System documentation & developer onboarding guide
```

Part 8 — Review Summary

The design is sound and internally consistent across both source documents. Rather than isolating the review in an appendix, the specific recommendations are now placed inline, next to the architecture they concern (look for the amber callout boxes in Parts 1, 4, and 5). This page is a quick index.

8.1 Inline Recommendations — Index

Location	Recommendation
Part 1.4 — Storage Strategy	Schedule recurring restore drills for Velero and Postgres PITR
Part 3.5 — Blast-Radius Protection	Unify the blast-radius threshold into one configurable numeric rule
Part 3.5 — Blast-Radius Protection	Add IP-based rate limiting on the unauthenticated ZTP endpoint
Part 3.5 — Blast-Radius Protection	Confirm idempotent handling of redelivered streaming/Celery events
Part 4.1 — Hybrid Southbound Automation	Add dedicated test coverage for pipeline Stages 2 and 3, and southbound drivers
Part 5.3 — Security & RBAC	Adopt explicit secrets management (Sealed Secrets / Vault)
Part 5.3 — Security & RBAC	Add Kubernetes NetworkPolicies restricting direct DB access
Part 5.3 — Security & RBAC	Design roles as permission sets now, to ease the move to granular RBAC

8.2 Suggested Implementation Sequence

5. Stand up the RKE2 cluster via Rancher; install Longhorn and MinIO from the Helm catalogue.
6. Deploy the Postgres operator (CloudNativePG recommended for simplicity) and Redis Sentinel.
7. Containerize the existing application logic (main.py, gnmi_discovery.py, config_lifecycle.py) into separate images per Deployment.
8. Wire KEDA for Celery worker autoscaling on Redis Streams queue depth.
9. Set up Velero and Postgres PITR backups — before any data of consequence exists, even in sandbox — and schedule the first restore drill.
10. Apply the secrets management and network policy recommendations as part of initial cluster hardening, not as a later pass.
11. Keep PNETLab on its dedicated VM; revisit Harvester migration only once the application cluster is stable.
12. Tackle the functional roadmap in the order given in Part 6: compliance scoring/reporting first, then BGP peering, then advanced topology protocols only if needed.

This document consolidates the Enterprise SDN Controller Blueprint and the Architecture de Déploiement & Fonctionnalités reference into a single English-language working document, with an added review section to support architecture sign-off before implementation begins.